COSC300 SDD

GROUP 20

Project Title

Smart Property Maintenance Tracker (AI-Powered Intelligent Maintenance Management Platform)

Group Members:

- ***MAREMENI T 202222839***
- ***NTULI SS 240004810***
- ***BADA ST 202308784***
- ***MOKGOSANA ST 202328708***
- ***MAKHUBU T 202391410***
- ***MNGUNI T 202219584***
- ***KHOZA SR 202056687***

REVISION HISTORY

To be completed by Coordinator

Date Received: _____/_____/_____

Signature: _____

Coordinator Notes:

_____

_____

_____

REVISION NUMBER	DATE	COMMENT
1.0	Date	Details

Table of Contents

2026 FINAL YEAR PROJECTS ..... ERROR! BOOKMARK NOT DEFINED.

DETAILED SYSTEM DESIGN DOCUMENT ..... ERROR! BOOKMARK NOT DEFINED.

NAME OF THE SYSTEM G ..... ERROR! BOOKMARK NOT DEFINED.

REVISION HISTORY **2**

TABLE OF CONTENTS **3**

1. DOCUMENT OVERVIEW **5**

Scope 5

Audience 5

Related Documentation ***Error! Bookmark not defined.***

Document Conventions 9

2. SYSTEM OVERVIEW **13**

Description ***Error! Bookmark not defined.***

System Architecture ***Error! Bookmark not defined.***

Software Architecture ..... ***Error! Bookmark not defined.***
Hardware Architectures ..... ***Error! Bookmark not defined.***

3. HARDWARE DESIGN **18**

Hardware Components 22

Computer Systems *25*
Peripherals *26*
Networks *27*
Project Specific hardware items (e.g. Sensors, Transducers, Robotics, Enclosure Design) *28*

Hardware Integration 28

Logical Design *28*
Physical Design *29*
Recovery Design *30*

4. SOFTWARE DESIGN 30

Software Packages34

{Software Module #1…N} 34

Software Integration 39

5. DATA AND DATABASE / FILES 42

Data Flow Diagrams 43

Database Design (ERD) 47

Files ***Error! Bookmark not defined.***

System Parameters 54

6. SYSTEM INTERFACES 55

{abc Interface} 58

7. SYSTEM PERFORMANCE 61

8. GLOSSARY / TERMINOLOGY 64

9. REFERENCES 64

1. DOCUMENT OVERVIEW

- document has been developed by GROUP 20 Ntuli SS, Khoza SR, Mnguni T, Bada ST, Makhubu TE ,Maremeni TJ, Mokgosana ST for SMART PROPERTY MAINTANANCE tracker for LI Khoza.

- This document was developed from project proposal, Agile SDLC blueprint, Software requirements specification (SRS) and This is intended to satisfy all the customer requirements, objectives and expectations.

scope

Hardware Environment:

- Cloud-Based Servers: The platform is hosted on Amazon Web Services (AWS), with all compute, storage, and AI services deployed within the af-south-1 (Cape Town) region for POPIA compliance.
- Client Devices: End-user smartphones (Android 10+, iOS 14+) and desktop computers (web browsers).

Software Environment:

- Frontend: Mobile Application developed using React Native (targeting Android and iOS from a shared codebase); Web Dashboard developed using React.js 18.
- Backend: Server framework Node.js 18 LTS with Express 4, deployed on AWS EC2.
- AI Components: AWS Comprehend (NLP text classification) and AWS Rekognition (image label detection and content moderation) both managed AWS AI services invoked via AWS SDK v3.
- Database: AWS RDS SSMS (primary relational store) for all structured data.

- Development & Deployment Tools: Agile development with GitHub for version control; CI/CD via GitHub Actions; Docker containers for consistent deployment environments.

Audience

1. **Software Developers and Engineers Skills Required:**

- Strong understanding of the Software Development Life Cycle (SDLC), particularly Agile methodologies
- Proficiency in programming languages such as JavaScript and Java
- Experience with modern frameworks such as React (web) and React Native (mobile)
- Knowledge of backend development and API integration
- Familiarity with cloud platforms such as AWS
- Ability to perform debugging, testing, and version control (e.g., Git) Assumptions:
- Developers have prior experience in building and maintaining software systems
- Developers have access to development tools, repositories, and documentation
- Developers are able to collaborate within an Agile team environment

---

2. **Tenants**

Skills Required:

- Basic ability to use mobile or web applications
- Ability to create, submit, and track maintenance tickets
- Basic digital literacy (e.g., logging in, navigating interfaces)

Assumptions:

- Tenants have access to internet-enabled devices (smartphone or computer)
- Tenants do not require technical knowledge of how the system works
- The system interface is user-friendly and easy to navigate

---

3. **Property Managers**

Skills Required:

- Ability to review, prioritise, and assign maintenance tickets
- Understanding of property management workflows
- Ability to interpret system-generated insights (e.g., AI suggestions or reports)
- Basic proficiency in using web and mobile applications Assumptions:
- Managers are familiar with administrative systems (not necessarily technical systems)
- Managers have consistent access to the system
- Managers are responsible for overseeing and coordinating maintenance operations

---

4. **Service Providers Skills Required:**

- Ability to view and respond to assigned maintenance tickets
- Ability to update job status (e.g., accepted, in progress, completed)
- Basic ability to use mobile/web applications Assumptions
- Service providers have access to internet-enabled devices
- Service providers are available to respond to job assignments in a timely manner

- Service providers are not required to understand the internal system architecture

5.System Administrator

Skills Required:

- Proficiency in database management systems (e.g., SQL Server, MySQL, PostgreSQL)
- Ability to write, optimize, and debug complex SQL queries (joins, subqueries, indexes)
- Understanding of database backup, recovery, and security protocols
- Knowledge of user access control and role-based permissions
- Ability to monitor system performance and troubleshoot issues
- Experience with API integrations and system interfaces
- Familiarity with cloud infrastructure and deployment pipelines
- Understanding of data integrity, constraints (primary keys, foreign keys), and normalization

Assumptions:

- System Administrator has advanced technical knowledge of the underlying database and application architecture
- Administrator has full access to system configuration settings, logs, and server resources
- Administrator is responsible for scheduled maintenance (e.g., backups, updates, index rebuilding) outside peak hours

---

Related documentation

-**Project proposal:** The document that provides the general overview of the project providing scope, objectives and the solution for problem in hand

-**Software requirements specifications** - A document for stating the functional and nonfunctional requirements, restrictions, and standard of quality the system must meet

-**Agile SDLC blueprint** - A guide to the iterative development process we follow in which each phase can be added on with new features along the development process

(b)Test plans and test results-

Test objectives-Ensures consistent, updated, good system perfomence validates system measures and also ensures user needs keep getting satisfied Different types of testing that can be used:

- **Unit Testing**: testing individual parts of the system to make sure they work correctly.
- **Integration Testing**: testing how different parts of the system work together
- **System Testing:** testing the whole system to make sure all features work correctly.
- **User Acceptance testing**: testing the system with real users to make sure it meets their needs.
- **Test results:**
- -Test reports are used to document defects detected and performance metrics
- -Results will be reviewed after each testing and improvement will be done if necessary

(c)Naming standards

Codebase Conventions:

- Variables and Functions: Use descriptive names in camelCase (e.g., assignServiceProvider, createTicket).
- Classes and Components: We use the pascalCase (e.g., MaintenanceManager, UserDashboard).
- Constants: Constants must be named in uppercases separated by underscores (e.g., MAX_RETRY_LIMIT).

Database naming conventions:

1. **Tables**: Must be in descriptive snake_case (e.g., user, service_provider).

2. **Columns**: Must be in descriptive snake_case (e.g., first_name, email_address). 3. Indexes and Keys: use clear prefixes (e.g., pk_user_id, fk_ticket_id). #APIs and Endpoints:

- Interface Control documents (ICDs)
- Interface (UI) Specifications Contents:
- Visual representations (wireframes/mockups) of system pages and features accessible to all users
- Layout structures for dashboards, ticket forms, and status tracking views Design

Guidelines:

- Standards for visual elements such as color schemes, typography, and icon usage
- Consistent layout and navigation patterns across web and mobile platforms

Relevance:

A well-designed user interface ensures usability and user engagement. It allows tenants, managers, and service providers to interact with the system efficiently. Advanced features such as AI-driven insights can further enhance usability by identifying and improving user interaction patterns.

2. API Interface Documentation Contents:

- API endpoints (URLs) and supported HTTP methods (GET, POST, PUT, DELETE)
- Request and response structures (e.g., JSON formats)
- Authentication and authorization mechanisms (e.g., tokens, API keys)
- Data models (schemas defining the structure of exchanged data)

Relevance:

Clear API documentation ensures seamless communication between frontend, backend, and external services. It enables efficient ticket submission, updates, and system integration. 3. Database Interface Documentation

Contents:

- Schema definitions describing the database structure (tables, attributes, and relationships)
- Query specifications for data retrieval and updates (CRUD operations)
- Data integrity constraints and validation rules

Relevance:

Defines how the system stores, retrieves, and manages data. Proper database interaction ensures data consistency, reliability, and efficient handling of maintenance tickets.

4. External Integration Document Contents:

- Description of integrations with external systems (e.g., cloud services, notification systems)
- Data mapping between internal system data and external services
- Error handling and exception management procedures
- Communication protocols and data exchange formats

Relevance:

Ensures smooth interaction between the system and third-party services. This is essential for features such as notifications, cloud storage, and scalability.

Document conventions

Unified model language is a standard visual language used to represent and design system in multiple system design in multiple graphical ways. UML is linked with object-oriented design and analysis. Diagrams can be classified as behavioral, structural and interaction Diagrammatic **notations used to represent architectural views in this document are:**

- Use case diagram- Visual way that show the interaction between actors (user or external system) and functions of system (use cases) provided by the system. Helps understand systems behaviour from users' perspective. It also captures functional requirements
  - Class diagrams- Visual representation of the structure of the system by displaying its classes, attributes, methods, and the relationship between them. Helps understand how components in the system interact and includes entities, attributes, methods and relationships
  - Component diagram- Visual representation of system structure by showcasing its various components and interactions. Displays how components of a system relate to one another

- Deployment diagram- Displays physical deployment of software components on hardware nodes. To map software artifacts to physical hardware nodes
- Activity diagram- Visualizes workflows, processes, or activities in a system. Shows the steps in how the system works

2. SYSTEM OVERVIEW

DESCRIPTION

The Smart Property Maintenance Tracker (SPMT) is a cloud-native, AI-powered property maintenance management platform that digitises and automates the complete lifecycle of maintenance requests for residential and commercial properties. SPMT is developed by Group 20 for the COSC300 module as a fully functional, production-ready application.

Key Components and Concepts:

User Profiles:

1. Tenants: Profiles include personal details, unit assignment, and notification preferences. Tenants submit maintenance tickets, track ticket status, and rate service providers after job completion.
2. Property Managers: Profiles include managed properties and units. Managers review AI classification results, override when necessary, assign service providers, monitor SLA compliance, and generate reports.
3. Service Providers: Profiles include company details, specialisation skills, GPS location, availability status, and performance rating. Providers accept or decline jobs and update job status in real time.
4. System Administrators: Have full system access including account management, AI performance monitoring, SLA configuration, and access to the immutable audit log.

AI-Powered Multi-Modal Conflict Resolution Engine:

The central innovation of SPMT is its Multi-Modal AI Conflict Resolution Engine, which independently analyses both the text description (via AWS Comprehend) and uploaded photographs (via AWS Rekognition) of each maintenance ticket. The engine compares the two classification outputs and applies a deterministic five-step arbitration algorithm when conflicts are detected. This architecture eliminates the critical failure mode where a tenant photographs a gas leak but describes it as an electrical fault — a single-signal system would misclassify and assign the wrong provider, potentially leaving a life-threatening hazard unaddressed.

SLA Management and Intelligent Routing:

SPMT enforces configurable Service Level Agreements (SLAs) per priority level, with automatic escalation on breach. The intelligent routing engine ranks available service providers by a composite score (50% performance rating, 30% GPS proximity, 20% current workload) and

surfaces the top three candidates to the Property Manager. Emergency tickets are auto-assigned without waiting for manual selection.

Immutable Audit Trail:

Every state change to every ticket, including AI classifications, overrides, assignments, and SLA events, is permanently recorded in an append-only audit log with actor, timestamp, previous state, new state, and comment. No audit record may be deleted or modified.

MAJOR EXTERNAL INTERFACES

Third-Party Integrations:

- AWS Comprehend: Natural Language Processing service for ticket text classification.
- AWS Rekognition: Computer vision service for image label detection and unsafe content moderation.
- AWS S3: Object storage for ticket photo attachments and exported PDF reports.
- AWS SNS: Push notification and SMS delivery infrastructure.
- AWS SES: Transactional email delivery (verification, approval, escalation, reports).
- AWS RDS SQL Server: Primary relational data store for all system entities.
- Google Maps SDK (react-native-maps): Service provider proximity display on mobile map.

External Data Sources:

- AWS Secrets Manager: Manages all API keys, database credentials, and application secrets. No secrets are hardcoded in source code or committed to version control.
- AWS CloudWatch: System monitoring, structured log aggregation, and alerting for API errors and performance anomalies.

SYSTEM ARCHITECTURE

2.1 High-Level Overview

User Interface Layer:

5. Components: React Native Mobile Application (iOS and Android); React.js Web Dashboard (all modern browsers).
6. Functionality: User registration and authentication; ticket submission and photo upload; real-time ticket status tracking; role-specific dashboards (tenant, manager, provider, admin).

Application Layer:

7. Components: Node.js/Express API Server (MOD-001 to MOD-010); Multi-Modal AI Conflict Resolution Engine (MOD-004); SLA Management Engine (MOD-005); Intelligent Provider Routing Engine (MOD-006); Notification Service (MOD-008).
8. Functionality: Processing and validation of all API requests; orchestration of AI classification, conflict resolution, and routing; enforcement of business rules and RBAC; SLA deadline tracking and automated escalation.

Data Access Layer:

9. Components: SQL Server on AWS RDS Multi-AZ for structured data; AWS S3 for file objects (images, PDFs); Repository pattern classes for all entity access.
10. Functionality: Secure storage and retrieval of all user, ticket, property, audit, and notification data; mssql connection pooling; parameterised queries only.

Integration Layer:

11. Components: AWS Application Load Balancer (ALB); AWS SDK v3 adapters for Comprehend, Rekognition, S3, SNS, SES, Secrets Manager; Socket.IO for real-time WebSocket communication.
12. Functionality: Distributes traffic across API server instances; manages all outbound calls to AWS managed services; delivers real-time dashboard updates via WebSocket.

Infrastructure Layer:

13. Components: AWS EC2 Auto Scaling Group (API servers); AWS VPC with public and private subnets; AWS Route 53 for DNS; AWS CloudWatch for monitoring; GitHub Actions CI/CD pipeline.
14. Functionality: Provides a scalable, highly available infrastructure; handles auto-scaling at 70% CPU threshold; ensures 99.5% uptime via Multi-AZ RDS and EC2 health checks.

Interfaces to External Systems:

- Hardware and Peripherals: SPMT is primarily cloud-based. Users access the platform via standard smartphones (camera + GPS required for full functionality) and desktop computers with internet connectivity. No specialised hardware is required.
- Systems Integration: SPMT integrates with AWS managed services (Comprehend, Rekognition, S3, SNS, SES, Secrets Manager) via AWS SDK v3 over HTTPS. Google Maps SDK is integrated for proximity display. All external service calls are wrapped in adapter classes with exponential backoff retry logic.

SOFTWARE ARCHITECTURE

2.2 Architectural Overview

SPMT adopts a three-tier client-server architecture with a stateless RESTful API at its core. The architecture enforces separation of concerns through a layered backend design:

Presentation Layer:

- Mobile Application: Developed using React Native (shared codebase for iOS and Android), providing a consistent user experience on mobile devices. Expo is used for build tooling.
- Web Application: Developed using React.js 18, accessible via modern web browsers on various devices, providing the Property Manager and System Administrator dashboards.

Business Logic Layer:

- Multi-Modal AI Conflict Resolution Engine: The core differentiator — independently invokes AWS Comprehend (text) and AWS Rekognition (images), applies the five-step conflict resolution algorithm, and produces a final classification within 5 seconds.
- Authentication & Authorisation Services: JWT-based stateless authentication with bcrypt password hashing (work factor ≥12). RBAC enforced at the Express middleware layer for every endpoint.
- SLA Management Service: Background cron job polling every 5 minutes to detect impending SLA breaches and trigger escalation notifications.

- Provider Routing Engine: Composite scoring algorithm ranking providers by performance rating (50%), proximity (30%), and current workload (20%).

Data Access Layer:

- SQL Server on AWS RDS: Stores all structured data — users, properties, units, tickets, audit logs, AI inference logs, notifications, and ratings. Repository pattern with parameterised queries.
- AWS S3: Object storage for ticket image attachments and exported PDF reports. Access exclusively via time-limited pre-signed URLs.

Integration Layer:

- API Gateway & Load Balancer: AWS ALB terminates HTTPS connections and distributes traffic across EC2 API server instances using round-robin with health checks.
- AWS Adapter Services: Each AWS SDK dependency (Comprehend, Rekognition, S3, SNS, SES, Secrets Manager) is wrapped in a dedicated adapter class, isolating business logic from third-party SDKs.
- Real-Time Updates: Socket.IO (WebSocket) delivers real-time ticket status changes to the web dashboard. Mobile clients receive updates via AWS SNS push notifications.

Summary of Software Architectures:

- Presentation Architecture: Focused on delivering intuitive, role-optimised interfaces through React Native mobile and React.js web applications.
- Business Logic Architecture: Centres on the AI-driven conflict resolution engine, SLA management, intelligent routing, and secure, role-based user management.
- Data Access and Storage Architecture: Utilises SQL Server for all structured data and AWS S3 for file objects, providing a reliable, POPIA-compliant data foundation.
- Integration Architecture: Employs the AWS SDK adapter pattern for all external service integrations, with the ALB providing load balancing and HTTPS termination.

HARDWARE ARCHITECTURES

2.3 Platforms

Cloud Infrastructure (Primary):

- Virtual Machines: EC2 instances (t3.medium minimum) in an Auto Scaling Group (min: 2, max: 10 instances) deployed on AWS in the af-south-1 region.
- Auto-Scaling & Load Balancing: AWS ALB distributes traffic across EC2 instances. Auto-scaling triggers at 70% CPU utilisation to add capacity before saturation.
- Managed Database: AWS RDS SQL Server Multi-AZ for automatic failover and 99.95% database availability.

2.4 Networks

Virtual Private Cloud (VPC):

- SPMT is deployed within an AWS VPC with two subnet tiers: Public Subnet (ALB, NAT Gateway) and Private Subnets (EC2 API servers, RDS). No application server or database is directly accessible from the internet.
- Firewalls & Security Groups: EC2 security group accepts traffic only from the ALB security group on port 443. RDS security group accepts traffic only from the EC2 security group on port 1433.

Networking & Security:

- End-to-end encryption: TLS 1.2+ for all client-API communication (enforced at ALB); TLS for all API-to-AWS-service communication (managed by AWS SDK).
- Role-Based Access Control (RBAC): Enforced at the Express middleware layer — every API endpoint is protected by role validation before reaching any business logic.

2.5 Peripherals

User Devices:

- Camera: Required for photo capture (ticket submission and job completion photos). React Native accesses the device camera via the expo-image-picker library.
- GPS / Location Services: Required for proximity-based provider routing. System degrades gracefully if permission is denied.
- Displays & Keyboards / Touch Input: Standard consumer smartphones and desktops. Minimum 360dp display width on mobile; 1280px on desktop browser.

2.6 Hardware Integration Summary

Compute Resources & Data Storage:

- SQL Server on AWS RDS: Stores all structured relational data with automated daily snapshots and 30-day retention.
- AWS S3: Stores ticket images and PDF reports with 11-nines (99.999999999%) durability and server-side AES-256 encryption.

Network Topology & Connectivity:

- Redundant Links: AWS Multi-AZ deployment across multiple availability zones within af-south-1 minimises downtime. RDS failover completes in under 2 minutes.

Scalability & Recovery:

- Auto-Scaling Policies: EC2 Auto Scaling Group automatically adds and removes instances based on CPU utilisation metrics, ensuring consistent performance at peak loads.

Integration Requirements:

- AWS SDK & Adapters: Each service (Comprehend, Rekognition, S3, SNS, SES, Secrets Manager) communicates over secure HTTPS using the AWS SDK v3, governed by IAM roles with least-privilege policies.

3. HARDWARE DESIGN

1. **internal system hardware components**

a) Cloud Servers and Virtualization:

- Our platform is hosted on cloud infrastructure AWS (amazon web service). We utilize virtual machines and container orchestration platforms (such as Kubernetes) to dynamically scale compute resources.

b) Networking Infrastructure:

- The architecture of the system includes Virtual Private Clouds (VPCs), load balancers, and secure VPNs.

c) Storage Systems

- Relation database (PostgreSQL) which stores users, tickets and properties
- NoSQL database to store real time ticket update and
- Cloud storage AWS S3 which stores uploaded images and AI dataset logs

These are the core elements needed to run the system:

d) Application servers will:

- Host backend logic, APIs, and AI processing requests
- Handle ticket classification routine and user management

e) Database servers will:

Store:

- User accounts (tenants, managers, and service providers)
- Maintenance tickets
- AI classification results
- Images and metadata

f) AI processing infrastructure will be responsible for:

- Cloud-based computations
- Text analysis using AWS Comprehend and
- Image analysis using AWS Rekognition

g) System will utilize User devices such as

- Smartphones (Android/ios)
- Laptops/desktops (windows, Linux macOS etc...)

These are used by:

- *Tenants, property managers and service providers*

2. Hardware design criteria

Consists of:

a) Scalability

- Which supports growing number of users, properties and tickets
- And support high ticket volume

b) Performance

- Where the AI ticket classification is completed within five seconds

c) Reliability

- High uptime for real time notifications and emergency handling

d) Security

- Secure storage and transmission of user data and images

e) Availability

- There will be 24/7 system access for tenants, property managers and service providers

3. Design approach

a) Cloud based architecture

- Is the AWS (amazon web service)

b) Distributed system design

- Frontend servers are the web application server (react based), mobile app interface (React Native) CDN (content delivery network)
- Backend servers are the APIs gateways, Application servers (Node js/ Express)
- AI processing modules analyze ticket description, images and assign categories

c) Micro-services approach

- independent services for ticket management, AI classification, notifications, and routing logic

4. Customization requirements

a) property level configuration

- Hardware scaling per number of properties and units

b) AI model customization

- Training datasets are stored and updated on servers
- Hardware which supports retraining processes

c) Notification preferences

- Devices must support push notifications and SMS

5. Environmental requirements

a) data centre /cloud environment

- Temperature controlled environments
- Redundant power supply

b) Network environment

- stable Internet connectivity needed
- Low latency for the real-time updates

c) User environment

- Mobile friendly system
- Works in multiple networks conditions (Wi-Fi/mobile data)

6. Target location

a) Primary deployment

- Cloud regions close to users

b) End Users

- At residential and commercial properties
- And Property managers across multiple locations

7. Physical dimension

a) cloud infrastructure

- No physical footprint for users

b) Uses devices like:

- Standard smartphones and laptops
- No specialized hardware needed

c) Optional (when system is extended)

- IoT sensors

8. Configuration parameters

a) Server configuration: minimum requirements for the system to operate efficiently (hardware components)

b) AI processing where AI models will operate independent distributed services responsible for intelligent automations

c) Network configuration

- Network components like routers, switches, API gateways, Load balancers are utilized by the system
- Network capabilities that determine rate of data transfer
- Protocol used for system communication
- Security protocols for secure communication

9. Integration Requirements:

- Cloud service integration
- Notification system
- Geolocation services
- Front end integration

10. Additional hardware design considerations

a) Backup and recovery systems

- automated backups

- Disaster recovery infrastructure

b) Monitoring hardware

- Server monitoring systems
- Performance tracking tools

c) Load handling

- Autoscaling servers during peak usage
- Queue system handling tickets spikes

d) Future expansion

- support for IoT integration
- Edge device for real time monitoring

Hardware Components

The system will adopt hybrid hardware approach which is the client-side, devices cloud infrastructure and supporting network hardware

Core hardware components:

1. ***End -user hardware***

- Smartphones (android/iOS)
- Tablets and laptops
- Desktop computers

Tenants, service providers, and property managers will use these devices to interact with the system

2. ***Cloud infrastructure***

- Virtual servers hosted on the cloud platform (e.g., AWS EC2)
- GPU-enabled instance for AI processing (imagine and text classification)
- distributed storage systems like AWS s3 (e.g., storage for images)

3. ***Database servers***

- Managed by relational databases (e.g. AWS RDS, S3)
- No SQL database is for unstructured data

b) customization

- AI workloads are improved using GPU enable virtual machines (VMs).
- Storage is customized to handle image uploads (up to 10MB)
- Auto-scaling groups dynamically give hardware resources based on demand

c) environmental and requirements

***Cloud servers run in a controlled data center with*:**

- redundant power supply
- Cooling systems
- Physical security

user's devices work:

- in a typical residential environment
- office environment

d) target location

- Regionally distributed
- Residential properties and service providers locations around the country

e) dimensions

- Cloud-based systems like AWS (amazon web service)
- Mobile and desktop devices

f) configuration parameters

- CPU: multicore processor (> i4)
- RAM: minimum of 8GB for AI processing

- Storage: starts from 100GB and upwards
- Network: minimum 100Mbps bandwidth for servers

g) integration requirements

Seamless integration with:

- AI services like AWS Comprehend, Rekognition (text and image processing)
- Notification systems (SMS, Email, Push Notification)
- Mobile and web applications

COMPUTER SYSTEMS

a) Platform tapes

- Cloud base virtual machines (VMs)
- Cloud side platforms (Android, iOS, and windows/mac OS)

b) Hardware standards and buses

- Standard cloud virtual platforms
- PCIe buses for GPU acceleration in AI modes

c) Operating systems

- Server: amazon Linux
- Users: Android, iOS, Windows, macOS

d) System utilities

- Containerization: Docker
- Orchestration: Kubernetes
- Monitoring tools: Cloud Watch or equivalent

e) Availability requirements

High availability:

- Multi-zone deployment

- Load balancing across servers

MTTR (mean time to repair):

- Target: < 1 hour (automatic recovery)

MTBF (mean time between failure)

- High due to cloud redundancy

Fault tolerance

- Replicated services and databases
- Failover mechanisms

Hot standby:

- Back up servers ready to take over immediately

Power backup:

- UPS and generator support at data centers

f) Performance requirements

- AI processing time is < 5 seconds per ticket
- Memory optimization for handling concurrent users
- High speed I/O for image uploads and retrieval

COMPUTER SYSTEMS

g) Platform tapes

- Cloud base virtual machines (VMs)
- Cloud side platforms (Android, iOS, and windows/mac OS)

h) Hardware standards and buses

- Standard cloud virtual platforms
- PCIe buses for GPU acceleration in AI modes

i) Operating systems

- Server: amazon Linux
- Users: Android, iOS, Windows, macOS

j) System utilities

- Containerization: Docker
- Orchestration: Kubernetes
- Monitoring tools: Cloud Watch or equivalent

k) Availability requirements

High availability:

- Multi-zone deployment
- Load balancing across servers

MTTR (mean time to repair):

- Target: < 1 hour (automatic recovery)

MTBF (mean time between failure)

- High due to cloud redundancy

Fault tolerance

- Replicated services and databases
- Failover mechanisms

Hot standby:

- Back up servers ready to take over immediately

Power backup:

- UPS and generator support at data centers

l) Performance requirements

- AI processing time is < 5 seconds per ticket
- Memory optimization for handling concurrent users
- High speed I/O for image uploads and retrieval

PERIPHERALS

a) Users side peripheral

- Cameras (for image uploads in maintenance ticket) *input device*

- Touchscreen (mobile devices) *input device*
- Keyboard and mouse (desktop users) *input device*
- Screen and mobile push notification *output device*

b) Server-side peripheral

- Network interface cards (NICs)
- Storage devices (SSD-based cloud storage)

c) Standards and interfaces

- USB (Users devices)
- Ethernet (server communication)
- Cloud storage APIs instead of traditional SAN/NAS

NETWORKS

a) Network components

- routers and switches within cloud infrastructure
- API gateways for request routing
- Load balancers for traffic distribution

b) Network capabilities

Data rates:

- 1Gbps ensuring rapid data transfer between cloud components and end users
- Internet-based access for clients (variable)

c) Protocols Used

- HTTP/HTTPS (for web communication)
- REST APIs (for application communication)
- TCP/IP (for network communication)
- Web socket (real-time updates)
- SMTP (email notifications)
- SMS gateway protocol

d) Security protocol

- SSL/TLS encryption
- VPN (for admin access)

Network configuration

- The system uses role-based access control (RBAC)

PROJECT SPECIFIC HARDWARE ITEMS (E.G. SENSORS, TRANSDUCERS, ROBOTICS, ENCLOSURE DESIGN)

The system interacts with hardware through the users' devices

a) sensors and input devices
- Smartphone cameras (for image-based issue detection)
- GPS/location services (for service provider proximity calculations)

b) AI processing hardware

GPU enabled cloud instances for:

- Image recognition (damage detection)
- Natural language processing (ticket classification)

c) Enclosure design
- Not applicable (cloud-based system)
- Mobile and desktop devices act as a user interaction endpoint

Hardware Integration

LOGICAL DESIGN

The logical Architecture follows a multi distributed model

a) Presentation layer
- Mobile apps and web interfaces

b) Application layer
- API service handling business logic

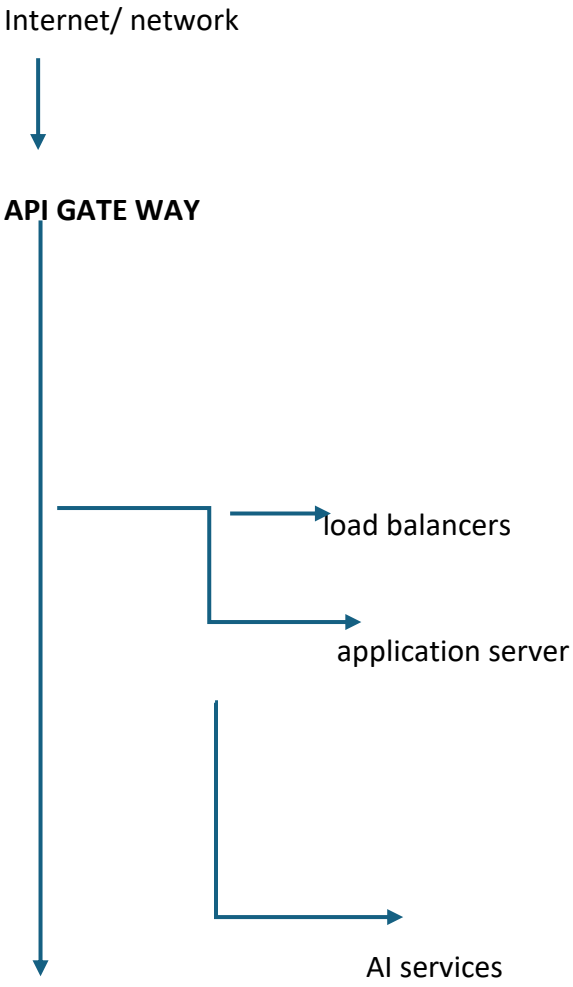
c) Data layer
- Databases and storage systems

d) Integration layer
- external services (AI API and notification systems)

e) **Topology**

Users' devices (mobile app)





PHYSICAL DESIGN

The physical integration is entirely cloud based

a) Cabling and connectivity

- High speed Fiber connections within data centers
- Internet connectivity for client devices

b) Connectors

- Standard ethernet interfaces in data centres
- Wireless (Wi-Fi/4G/5G) for users' devices

c) Deployment layout

- Distributed across multiple availability zones
- Redundant network paths for reliability

RECOVERY DESIGN

The physical integration is entirely cloud based

a) Cabling and connectivity
- High speed Fiber connections within data centers
- Internet connectivity for client devices

b) Connectors
- Standard ethernet interfaces in data centers
- Wireless (Wi-Fi/4G/5G) for users' devices

c) Deployment layout
- Distributed across multiple availability zones
- Redundant network paths for reliability

4. SOFTWARE DESIGN

SOFTWARE COMPONENTS AND ARCHITECTURE

SPMT's backend is structured into 10 named software modules (MOD-001 to MOD-010) within a Node.js monorepo. Each module encapsulates a specific area of business logic with well-defined inputs, outputs, and dependencies. The architecture follows the Repository pattern for data access, an Adapter pattern for all AWS SDK integrations, and an Express middleware pipeline for security enforcement.

Key Software Components:

COMPONENT	DESCRIPTION
Frontend (UI/UX)	React Native mobile app and React.js web dashboard providing role-specific, intuitive interfaces.
API Server (Backend)	Node.js 18 LTS + Express 4 handling authentication, AI orchestration, routing, SLA management, and data processing.
Multi-Modal AI Engine	AWS Comprehend (text NLP) + AWS Rekognition (image analysis) with a deterministic five-step conflict resolution algorithm.
Database Services	AWS RDS SQL Server for all structured relational data with mssql connection pooling and Repository pattern.
File Storage	AWS S3 (af-south-1) for ticket image attachments and PDF reports, accessed via time-limited pre-signed URLs only.

Authentication Service	JWT-based stateless authentication; bcrypt password hashing (work factor ≥12); RBAC enforced at Express middleware layer.
Notification Service	AWS SNS (push + SMS) and AWS SES (email) for all system-generated notifications, with exponential backoff retry.
CI/CD Pipeline	GitHub Actions for automated testing, Docker image building, and deployment to AWS Elastic Beanstalk.
Analytics & Monitoring	AWS CloudWatch for structured log aggregation, API error alerts, and performance metric dashboards.

COTS SOFTWARE AND CONFIGURATIONS

COTS SOFTWARE	PURPOSE	CONFIGURATION
Node.js 18 LTS	RESTful API server runtime	Stateless Express 4 middleware; JWT authentication; rate limiting via express-rate-limit.
React Native	Mobile application framework	Expo build tooling; react-native-maps for provider proximity; expo-image-picker for camera.
React.js 18	Web dashboard framework	Socket.IO client for real-time updates; Axios for API calls; React Router v6.
SQL Server (AWS RDS)	Primary relational database	Multi-AZ deployment; mssql connection pooling (min:5, max:20); TLS enforced (rds.force_ssl=1).
AWS SDK v3	AWS service integrations	Adapters for Comprehend, Rekognition, S3, SNS, SES, Secrets Manager. Exponential backoff: max 3 retries.
Socket.IO 4	WebSocket real-time updates	WSS connections; JWT auth on handshake; rooms scoped by managerID.
bcrypt	Password hashing	Work factor ≥12. Plaintext passwords never stored or logged.
jsonwebtoken	JWT token management	Access token TTL: 24h. Refresh token TTL: 7d. Algorithm: HS256.
Multer	File upload middleware	Multipart/form-data handling for ticket image uploads (max 10MB per file, JPEG/PNG only).
PDFKit / Puppeteer	PDF report generation	Generates report PDFs on demand, stored to S3 with pre-signed download URLs.
Jest + Supertest	Unit and integration testing	80% statement coverage target. Mocked AWS adapters for unit test isolation.
Docker	Containerisation	Containers for API server; consistent environment across dev, staging, and production.
GitHub Actions	CI/CD pipeline	Automated lint → unit test → integration test → Docker build → AWS deploy on push to main.
AWS CloudWatch	Monitoring and logging	Structured JSON logs via Winston; retention: 30 days. Metric alarms for error rates and latency.
node-cron	Background job scheduler	SLA polling job runs every 5 minutes. Emergency auto-assignment triggered after 20 minutes.

Detailed Software Integration Design

Frontend-Backend Communication:

The system uses **RESTful APIs** to enable communication between the frontend (**React Native** mobile application and **React.js** web dashboard) and the backend (**Node.js** with **Express framework**).

API requests follow standard **HTTP methods** (**GET, POST, PUT, DELETE**) and exchange data in JSON format. All communication is secured using **JWT (JSON Web Token)-based authentication,** ensuring that only authorized users can access system resources. **HTTPS (TLS encryption)** is used to protect data in transit.

Database Integration:

- **Microsoft SQL Server (AWS RDS)** is used as the primary relational database. Stores structured data such as users, tickets, properties, and service providers
- Accessed using **Node.js** mssql driver with connection pooling
- Repository pattern ensures abstraction and maintainability
- Transactions are used for critical operations (e.g., ticket assignment)

AI Integration (Core System Feature):

The system integrates with AWS AI services:

- **AWS Comprehend:** Text classification
- **AWS Rekognition:** Image analysis

Processing Flow:

1. Ticket submitted by tenant
2. Text sent to Comprehend
3. Images sent to Rekognition
4. Results combined in backend
5. Final classification stored in SQL Server

Multi-Modal Conflict Resolution Engine:

To prevent incorrect classification:

- Text and image results are analysed independently
- If results conflict:
  - System flags the ticket
  - Priority escalated to emergency if risk detected
  - Property manager notified immediately

File Storage Integration

- AWS S3 is used to store:
  - Ticket images
  - Attachments
- Files are stored securely using:
  - Pre-signed URLs
  - Server-side encryption

Notification System Integration

The system uses:

- AWS SNS → Push notifications & SMS
- AWS SES → Email notifications

Used for:

- Ticket submission confirmation
- Assignment notifications
- Emergency alerts

Authentication & User Management

- JWT-based authentication system
- Role-Based Access Control (RBAC):
  - Tenant
  - Property Manager
  - Service Provider
  - Admin
- Passwords stored using hashing (bcrypt)

Logging & Monitoring

- AWS CloudWatch:
  - System logs
  - Error tracking
  - Performance monitoring
- Logs include:
  - API requests
  - AI classification results
  - Ticket updates

CI/CD PIPELINE:

STAGE	TOOLS	DETAILS
Development	GitHub + GitHub Actions	Developers push feature branches to GitHub, triggering automated CI pipeline on pull request.
Testing	Jest + Supertest	Runs unit tests (mocked AWS adapters), integration tests (test DB), and linting on every PR.
Building	Docker	GitHub Actions builds a Docker image of the API server and pushes to Amazon ECR (Elastic Container Registry).
Deployment	AWS Elastic Beanstalk	Zero-downtime rolling deployment to Elastic Beanstalk environment on merge to main. Rollback on health check failure.
Monitoring	AWS CloudWatch	Tracks API response times, error rates, AI pipeline latency, and SLA breach counts. Alarms notify SysAdmin.

Software Packages

{SOFTWARE MODULE #1…N}

The following sections describe each software module in detail, using a standardized card format specifying module name, purpose, implementation, execution location, inputs, outputs, dependencies, error handling, relationship to users, and interfaces.

MOD-001 Authentication & Authorisation Module:

Module Name	SPMT Auth
Purpose	Handles user registration, email verification, login, JWT token issuance, role-based access control (RBAC), account locking, and password reset.
Implementation	JavaScript (Node.js 18 / Express 4). bcrypt for password hashing (work factor ≥12). jsonwebtoken for JWT issuance and verification.
Execution Location	AWS EC2 API Server — runs in-process with the main Express application.
Inputs	Registration payload (name, email, phone, identityNumber, password, role); login credentials (email, password); JWT Bearer token; email verification token; password reset token.
Outputs	JWT access token (24h TTL), refresh token (7d TTL), HTTP 200/401/403/409 status codes, email dispatch requests to NotificationModule, account status updates in users table.
Dependencies	UserRepository, NotificationModule, bcrypt, jsonwebtoken, AWS Secrets Manager (JWT secret).
Error Handling	5 consecutive failed logins triggers account lock + unlock email (REQ-006). Expired tokens return HTTP 401. Insufficient role returns HTTP 403. Duplicate email returns HTTP 409. All auth events logged to security_audit_log (NFR-SEC09).
Relationship to Users	All four user roles interact with this module at login, registration, and token refresh. RBAC controls exactly what each role can access throughout the system.
Interfaces	POST /auth/register, POST /auth/login, POST /auth/refresh, POST /auth/logout, POST /auth/verify-email, POST /auth/reset-password
Related Requirements	REQ-001 to REQ-008, NFR-SEC01, NFR-SEC02, NFR-SEC03, NFR-SEC04, NFR-SEC09.

MOD-002 Property & Unit Management Module

Module Name	SPMT Property
Purpose	Enables Property Managers to create and manage properties and units, and assign tenants to specific units. Enforces the one-tenant-per-unit rule.
Implementation	JavaScript (Node.js 18 / Express 4).
Execution Location	AWS EC2 API Server.
Inputs	Property data (name, address, type); unit data (unit number, floor); tenant assignment request (managerID, tenantID, unitID).
Outputs	Property/unit UUIDs on creation (HTTP 201), updated unit-tenant mapping, HTTP 200/400/403/409.

Dependencies	PropertyRepository, UnitRepository, UserRepository.
Error Handling	Double tenant assignment returns HTTP 409. Non-manager access returns HTTP 403. Ticket submission blocked if tenant not assigned to unit (REQ-012, returns HTTP 403).
Relationship to Users	Used exclusively by Property Managers (and System Admins for oversight). Tenants are passively affected — they cannot submit tickets until assigned to a unit by their manager.
Interfaces	POST /properties, GET /properties, POST /properties/{id}/units, POST /properties/{id}/units/{unitId}/assign
Related Requirements	REQ-009 to REQ-012, BR-001.

MOD-003 Ticket Submission Module

Module Name	SPMT Ticket Submit
Purpose	Orchestrates the submission of maintenance tickets: validates input, stores ticket and photos (S3), triggers the AI classification pipeline, initialises the SLA clock, and sends confirmation notifications.
Implementation	JavaScript (Node.js 18). Multer middleware for multipart/form-data image upload. AWS SDK v3 for S3 upload.
Execution Location	AWS EC2 API Server.
Inputs	Ticket payload (title ≥5 chars, description ≥20 chars, urgency enum, unitID); up to 5 JPEG/PNG images (≤10MB each).
Outputs	Ticket UUID displayed to tenant (REQ-014); S3 image keys stored in ticket_attachments; AI pipeline triggered asynchronously; confirmation email/SMS to tenant (REQ-018).
Dependencies	TicketRepository, S3Adapter, AIModule (MOD-004), SLAModule (MOD-005), NotificationModule (MOD-008), AuditLogRepository.
Error Handling	Validation failures: HTTP 422 with inline field errors. Image content moderation rejection: HTTP 400 with reason code. S3 upload failure: exponential backoff retry (max 3, NFR-R02). Any failure prevents ticket storage (NFR-P05 — zero data loss).
Relationship to Users	Used exclusively by Tenants. Tenants must be assigned to a unit (BR-001) to access this module.
Interfaces	POST /tickets (multipart/form-data); GET /tickets (tenant's own tickets); GET /tickets/{id}
Related Requirements	REQ-013 to REQ-018, NFR-P05, BR-001, BR-008.

MOD-004 Multi-Modal AI Classification Module

Module Name	SPMT AI Engine
Purpose	The core AI module. Independently invokes AWS Comprehend (text) and AWS Rekognition (images), applies the five-step conflict resolution algorithm, and produces a final classification with an immutable AI inference log entry.
Implementation	JavaScript (Node.js 18). AWS SDK v3 (@aws-sdk/client-comprehend, @aws-sdk/client-rekognition). Async execution using Promise.all() for parallel image analysis.
Execution Location	AWS EC2 API Server — executed asynchronously after ticket submission.
Inputs	Ticket description text (string ≥20 chars); S3 image keys (string[], max 5); confidence threshold (default 0.60); emergency visual threshold (default 0.70).

Outputs	Final category (string); combined confidence score (0.0–1.0); conflictFlag (boolean); manualReviewFlag (boolean); priority (possibly escalated to Emergency); AI_INFERENCE_LOG records (one per AWS service call).
Dependencies	ComprehendAdapter, RekognitionAdapter, LabelCategoryMapper (labelCategoryMap.json), EmergencyKeywordScanner, AIInferenceLogRepository, TicketRepository, NotificationModule.
Error Handling	Comprehend unavailable: proceed with image-only. Rekognition unavailable: proceed with text-only. Both unavailable: ticket placed in Manual Review, SysAdmin alerted (NFR-R03). All errors trigger retry (max 3) before fallback.
Relationship to Users	Transparent to Tenants (automatic process). Property Managers see classification results, conflict badges, and confidence scores on the dashboard. SysAdmin monitors AI performance metrics.
Interfaces	Internal module — invoked by MOD-003 (TicketSubmission). Calls INT-003 (Comprehend) and INT-004 (Rekognition). Updates ticket record and audit log via TicketRepository and AuditLogRepository.
Related Requirements	REQ-019 to REQ-028, NFR-P02, NFR-S01, NFR-S02, BR-006, BR-008, BR-009, BR-010, CON-002, CON-006.

CONFLICT RESOLUTION ALGORITHM — FIVE STEPS

Step 1: Image emergency visual detected (confidence ≥0.70) → EMERGENCY escalation. Overrides text classification and tenant urgency. PM + all emergency providers notified within 30s.

Step 2: text category === image category → AGREE. Combined confidence = (0.4 × textConf) + (0.6 × imgConf). Route if combined ≥0.60.

Step 3: text ≠ image AND either confidence <0.60 → MANUAL REVIEW REQUIRED. PM notified to classify manually before routing.

Step 4: text ≠ image AND both ≥0.60 → CONFLICT DETECTED. Image classification used as primary. PM notified with both results side-by-side.

Step 5: AWS service unavailable → GRACEFUL FALLBACK. Available service proceeds; if both down → Manual Review. SysAdmin alerted.

MOD-005 SLA Management Module

Module Name	SPMT SLA Engine
Purpose	Assigns SLA deadlines at ticket creation and runs a scheduled background job (every 5 minutes) to detect impending breaches and trigger escalation notifications.
Implementation	JavaScript (Node.js 18). node-cron for scheduled polling. Background job runs in the same process as the API server.
Execution Location	AWS EC2 API Server — background cron thread.
Inputs	Ticket priority level (enum); ticket creation timestamp; SLA configuration from sla_config table.
Outputs	SLA deadline timestamps stored in ticket record; warning notification to PM at 75% of SLA elapsed; escalation notification to PM + SysAdmin on breach; SLA_BREACHED flag on ticket; audit log entry for every SLA event.
Dependencies	TicketRepository, SLAConfigRepository, NotificationModule, AuditLogRepository.
Error Handling	Emergency tickets unassigned after 20 minutes trigger auto-assignment (BR-003). SLA events logged even if notification delivery fails. SLA poll errors logged to CloudWatch with SysAdmin alert.

Relationship to Users	PM sees SLA countdown and breach flags on dashboard. SysAdmin monitors SLA compliance reports. Tenants receive escalation notifications when their ticket is overdue.
Interfaces	Internal module — invoked by cron scheduler. Reads from tickets and sla_config; writes to audit_log and notifications via repositories.
Related Requirements	REQ-029 to REQ-032, NFR-P03, BR-003.

MOD-006 Intelligent Provider Routing Module

Module Name	SPMT Router
Purpose	Selects and ranks the most suitable Service Providers for each classified ticket using a composite scoring algorithm. Handles auto-assignment for Emergency tickets and decline fallback chains.
Implementation	JavaScript (Node.js 18). Haversine formula for GPS distance calculation.
Execution Location	AWS EC2 API Server.
Inputs	Ticket category; property GPS coordinates; candidate provider pool (specialisation match + AVAILABLE/ON_CALL status); provider GPS locations, average ratings, and current workload counts.
Outputs	Top 3 ranked providers surfaced to PM with score, rating, and proximity (REQ-035). Auto-assignment record for Emergency tickets (REQ-036). Decline reason stored in audit log (REQ-038).
Dependencies	ServiceProviderRepository, TicketRepository, NotificationModule, AuditLogRepository, GeoDistanceCalculator.
Error Handling	No eligible providers: PM notified; Emergency tickets placed in Emergency Queue (NFR-S03). Provider declines: fallback to next-ranked candidate automatically (REQ-037). Off-duty providers excluded from candidate pool (BR-007).
Relationship to Users	PM sees top-3 recommendations with scores. Emergency providers receive auto-assignment notifications. Service Providers who decline trigger automatic re-routing.
Interfaces	GET /tickets/{id}/provider-recommendations, POST /tickets/{id}/assign, PATCH /tickets/{id}/accept, PATCH /tickets/{id}/decline
Related Requirements	REQ-033 to REQ-038, BR-002, BR-003, BR-007.

MOD-007 Ticket Lifecycle & Audit Trail Module

Module Name	SPMT Lifecycle
Purpose	Manages all ticket state transitions and maintains an immutable, append-only audit log. Enforces business rules around closed/rejected ticket modifications and valid state transitions.
Implementation	JavaScript (Node.js 18). SQL Server transactions for atomic state update + audit log INSERT.
Execution Location	AWS EC2 API Server.
Inputs	Ticket ID; requested new state; actor user ID and role; optional comment or justification string.
Outputs	Updated ticket record in DB; immutable audit log entry (INSERT only — no UPDATE or DELETE); notifications dispatched on state change.
Dependencies	TicketRepository, AuditLogRepository (INSERT only), NotificationModule.
Error Handling	Invalid state transitions: HTTP 409. Modification of Closed/Rejected ticket without reopen permission: HTTP 403. Missing justification on reopen: HTTP 422. All errors logged.

Relationship to Users	All roles interact with this module: Tenants track status; PMs manage workflow; Providers update job status; SysAdmin views full audit trail.
Interfaces	PATCH /tickets/{id}/status, POST /tickets/{id}/reopen, GET /tickets/{id}/audit-trail
Related Requirements	REQ-039 to REQ-042, BR-004, BR-005.

MOD-008 Notification Module

Module Name	SPMT Notify
Purpose	Delivers push notifications, email, and SMS to relevant actors for all significant ticket events. Provides retry logic for failed deliveries.
Implementation	JavaScript (Node.js 18). AWS SDK v3 (@aws-sdk/client-sns, @aws-sdk/client-ses).
Execution Location	AWS EC2 API Server.
Inputs	Recipient user ID or contact details; notification type (push/email/SMS); message payload; optional schedule timestamp.
Outputs	Notification dispatch record in notifications table; delivery status updated on AWS callback; retry queued on failure.
Dependencies	SNSAdapter, SESAdapter, NotificationRepository, UserRepository.
Error Handling	Exponential backoff retry: max 3 attempts (NFR-R02). Permanent failure: logged (deliveryStatus=FAILED), SysAdmin alerted if emergency notification fails. Users configure preferred channels (REQ-045).
Relationship to Users	All roles receive notifications. Tenants: ticket confirmation, status updates, provider assigned. PMs: AI conflict alerts, SLA warnings/breaches, emergency escalations. Providers: job assignments. SysAdmin: system alerts, low-rating provider flags.
Interfaces	Internal module — invoked by all other modules. sendPush(), sendEmail(), sendSMS(), sendEmergencyNotification(), retryFailed()
Related Requirements	REQ-043 to REQ-046, NFR-P03, NFR-S02, BR-003, BR-009.

MOD-009 Service Provider Performance & Rating Module

Module Name	SPMT Ratings
Purpose	Collects tenant ratings post-completion and maintains a running average per provider. Flags underperforming providers and feeds rating data into the routing algorithm.
Implementation	JavaScript (Node.js 18). Atomic UPDATE with running average calculation on SQL Server.
Execution Location	AWS EC2 API Server.
Inputs	Tenant user ID; ticket ID (must be in CLOSED state); provider ID; score (1–5 integer); optional comment string.
Outputs	PerformanceRating record inserted; ServiceProvider.averageRating updated atomically; low-rating alert to SysAdmin if average <2.0 over ≥5 tickets (REQ-050).
Dependencies	PerformanceRatingRepository, ServiceProviderRepository, NotificationModule.
Error Handling	Duplicate rating for same ticket: HTTP 409 (UNIQUE constraint on ticketID in performance_ratings). Rating on non-closed ticket: HTTP 400.
Relationship to Users	Tenants submit ratings after job closure. PMs see provider ratings on the assignment screen. SysAdmin monitors flagged underperforming providers.
Interfaces	POST /tickets/{id}/rating, GET /providers/{id}/profile, GET /providers/{id}/ratings
Related Requirements	REQ-047 to REQ-050.

MOD-010 Reporting & Analytics Module

Module Name	SPMT Reports
Purpose	Generates data-driven management reports on ticket volume, resolution times, SLA compliance, provider performance, and AI classification accuracy. Exports reports as PDF to S3.
Implementation	JavaScript (Node.js 18). PDFKit or Puppeteer for PDF generation. AWS SDK v3 for S3 upload.
Execution Location	AWS EC2 API Server (on-demand, triggered by dashboard request).
Inputs	Filter parameters (property ID, date range, category, provider ID, priority level). Requestor must have PM or SysAdmin role.
Outputs	Structured report data as JSON (dashboard rendering) and PDF file stored in S3; pre-signed download URL returned (REQ-056). Report types: Ticket Volume, Resolution Time, SLA Compliance, Provider Performance, AI Performance.
Dependencies	TicketRepository, AuditLogRepository, AIInferenceLogRepository, PerformanceRatingRepository, S3Adapter, PDFGenerator.
Error Handling	No data matching filters: HTTP 404 with message. S3 PDF upload failure: retry (max 3); if persistent, return JSON data only with error flag. PDF generation errors: logged to CloudWatch with SysAdmin alert.
Relationship to Users	PMs: access property-specific and provider reports via web dashboard. SysAdmin: access system-wide reports and AI performance analytics.
Interfaces	GET /reports/ticket-volume, GET /reports/resolution-time, GET /reports/sla-compliance, GET /reports/provider-performance, GET /reports/ai-performance
Related Requirements	REQ-051 to REQ-056, BR-006.

Software Integration

1. Frontend–Backend Communication

Technology Used:

- API Type: RESTful JSON API over HTTPS.
- Middleware: Express.js (Node.js), Multer (file uploads), express-rate-limit (rate limiting), CORS middleware.
- Authentication: JWT Bearer token in Authorization header. Tokens issued by MOD-001 on successful login.

Integration Flow:

- The frontend (React Native / React.js) interacts with the backend (Node.js/Express.js) via HTTPS API calls.
- All API requests (except /auth/register and /auth/login) must include a valid JWT Bearer token.
- Real-time dashboard updates are received by the React.js web dashboard via Socket.IO WebSocket connections (authenticated at handshake).

Configuration & Customisation:

- Custom Express middleware for error handling ensures all API failures return consistent JSON error envelopes: { error: string, code: string, details?: object }.

- Rate-limiting middleware (express-rate-limit): 100 requests/min per IP (unauthenticated), 1000 requests/min per user (authenticated).
- CORS middleware configured to allow requests only from registered client origins (mobile app and web dashboard domains).

2. Backend and Database Integration

Technology Used:

- Primary Database: AWS RDS SQL Server (af-south-1).
- Driver: mssql (tedious) with connection pooling (min: 5, max: 20 connections).
- Pattern: Repository pattern — each entity has a dedicated Repository class encapsulating all DB queries.

Integration Flow:

- All user, ticket, property, audit, AI inference, notification, and rating data is stored in SQL Server.
- State transitions (ticket status update + audit log INSERT) are executed within a single SQL Server transaction to ensure atomicity.
- Ticket data exceeding the connection pool capacity is queued transparently by the Node.js event loop; RDS supports up to 200 simultaneous connections on db.t3.medium.

Configuration & Customisation:

- All database queries are parameterised — no string concatenation (OWASP A03 SQL injection prevention).
- RDS parameter rds.force_ssl=1 enforces TLS on all database connections.
- Slow query logging enabled for queries >100ms, published to CloudWatch.

3. AI Model Integration (Comprehend & Rekognition)

Technology Used:

- Text AI Service: AWS Comprehend — custom classifier endpoint trained on maintenance ticket data.
- Image AI Service: AWS Rekognition — DetectLabels + DetectModerationLabels API operations.
- Deployment: Managed AWS services; no GPU instance or model server management required.

Integration Flow:

- Ticket text (title + description) is submitted to AWS Comprehend classifyDocument API. Response: [{Name: category, Score: confidence}].
- Each uploaded image (S3 key) is submitted to AWS Rekognition DetectModerationLabels (safety check) and then DetectLabels (classification). Moderation failures block the image from processing or storage.
- Results feed into the five-step conflict resolution algorithm (MOD-004) which produces the final classification, combined confidence, and flags.

Configuration & Customisation:

- AI confidence thresholds (AI_TEXT_CONFIDENCE_THRESHOLD=0.60, AI_EMERGENCY_VISUAL_THRESHOLD=0.70) are stored as system parameters, adjustable by SysAdmin without code changes.
- labelCategoryMap.json maps Rekognition label strings to SPMT maintenance categories. Updateable via CI/CD without schema migration.
- Comprehend and Rekognition calls are parallelised using Promise.all() where possible to meet the 5-second pipeline budget (NFR-P02).

4. Cloud Storage & File Handling

Technology Used:

- File Storage: AWS S3 (af-south-1) — two buckets: spmt-ticket-images-{env} and spmt-reports-{env}.
- Middleware: Multer (Node.js) for multipart/form-data parsing and in-memory buffering before S3 upload.

Integration Flow:

- Tenants upload images (JPEG/PNG, max 10MB each, max 5 per ticket) via the POST /tickets API endpoint.
- Multer parses and validates files in memory. AWS S3Adapter uploads each file to the ticket-images bucket under key: images/{ticketID}/{UUID}.{ext}.
- S3 objects are accessed exclusively via time-limited pre-signed URLs (TTL: 15 minutes for images, 24 hours for reports) generated by the API on request. No S3 object is ever publicly accessible.

Configuration & Customisation:

- Custom file naming: each uploaded file receives a UUID-based key to prevent naming collisions and obscure original filenames.
- Automated file format validation: Multer file filter rejects any file with MIME type other than image/jpeg or image/png.
- S3 bucket policy enforces Block Public Access. Server-side encryption: SSE-S3 (AES-256).

5. CI/CD Pipeline Integration (Automation & Deployment)

Technology Used:

- Version Control: GitHub (feature branch workflow; protected main branch requires PR review + all CI checks passing).
- CI/CD Tools: GitHub Actions for automated pipelines; Docker for container image builds.

Hosting: AWS Elastic Beanstalk (Docker platform) for zero-downtime rolling deployments.

Configuration & Customisation:

- Custom Dockerfiles define lightweight Node.js 18-alpine containers for the API server, reducing image size and attack surface.
- Automated rollback scripts in GitHub Actions revert to the previous Docker image if the Elastic Beanstalk health check fails within 5 minutes of deployment.

- Environment-specific configuration (dev, staging, prod) managed via AWS Secrets Manager — no secrets in GitHub Actions environment variables.

6. Notification System Integration

Technology Used:

- Push Notifications: AWS SNS (mobile platform endpoints for iOS APNs and Android FCM).
- SMS: AWS SNS SMS API (E.164 phone number format).
- Email: AWS SES (DKIM-signed transactional email from noreply@spmt.co.za).

Integration Flow:

- Users receive real-time push notifications via AWS SNS platform endpoint ARNs registered at mobile app install.
- Emergency alerts are sent as highest-priority SNS messages (TTL=0, ensuring immediate delivery without queuing).
- Transactional emails (verification, approval, escalation, report download) are sent via AWS SES with DKIM signing to prevent spoofing.

Configuration & Customisation:

- Custom notification templates per event type ensure personalised, contextual messages for each actor role.
- Push notifications are rate-limited per user to avoid notification fatigue (max 10 push notifications per user per hour for non-emergency events).

Email logs and SMS delivery receipts are stored in the notifications table with deliveryStatus tracking for debugging and audit purposes.

5. DATA AND DATABASE / FILES

Overview of Software Modules to Data / Repositories Linkages

The smart Property Maintenance Tracker system employs a hybrid database architecture combining Relational Database System and File Storage to ensure efficient storage, fast retrieval, and scalable management property maintenance data.

1.**Relational Database System**

- Maintain relationships between users, properties, and maintenance requests
- Ensure data consistency and integrity
- Support efficient querying and reporting

- **Database Management Tool**:
  The relational database is managed using tools such as SQL Server Management Studio (SSMS)

  SSMS is used for:
  - Creating and managing database tables
  - Writing and executing SQL queries
  - Monitoring and maintaining the database

2**.File Storage (AWS S3 / Local Server)**

- Stores multimedia and unstructured maintenance data such as:
  - Images of property damage (e.g., leaks, broken fixtures)
  - Maintenance report and invoices (PDFs)
  - Service provider completion evidence and inspection documents
- Ensures secure and scalable storage of large files
- Links stored files to specific maintenance request records

Data Flow Diagrams

Data Flow Between Modules and Repositories

- User Authentication and Profile → System Authentication module + Relational Database

  - Stores tenant, property manager, and service provider profiles
  - Manages secure login credentials and role-based access control
  - Ensures encrypted authentication tokens and session handling

- Maintenance Requests and Property Records → Relational database

  - SQL stores structured maintenance request details (property ID, issue type, priority level)
  - Ensures both consistency and real-time responsiveness

- Image Uploads and Maintenance Evidence → File Storage

- Stores images property issues
- Links media files to maintenance request IDs
- Allows service providers to upload proof of completed work
- Ensures secure retrieval of files when reviewing maintenance history

➢ Analytics and System Logs → Relational database

- Tracks maintenance frequency per property
- Logs system activity such as request creation, updates, and completion
- Supports performance dashboards of property managers
- Enables reporting on response times and technician efficiency

Security and Data Management

- Data Encryption: All data is encrypted in transit (TLS) and rest (AES-256)
- Access Control: Role-Based Access Control (RBAC) restricts data access based on user type (tenant, owner, technician, admin)
- Backup and Recovery: Automated daily backups with cloud redundancy unsure data safety
- Audit Trails: All maintenance activities are logged for accountability and tracking

Data Flow Diagrams (DFDs)

1. Top-Level DFD (Context Diagram)

Summary:

The user (tenant, property manager, or service provider) interacts with the Smart Property Maintenance Tracker to perform actions such as reporting maintenance issues, tracking requests, uploading evidence, and communicating with other users.

The system processes these requests and retrieves/ store data from databases and file storage.

External system (e.g., notification services, IoT sensors if applicable) may also interact with the platform.

<u>Entities and Flow</u>:

Actors:

- Tenant
- Property Manager
- Service provider

Processes:

- User authentication
- Maintenance request submission
- Work order assignment
- Maintenance tracking

Data Stores:

- Relational Database + SSMS
- File Storage

2. System-Level DFD (Level 1 DFD)

- User Interactions
  - User logs in → System authentication
  - Tenant reports issue → Data stored in relational database
  - User tracks request → Data retrieved from relational database
  - User uploads images → Stored in File Storage

- Service Provider Interactions
  - Receives assigned maintenance task → Retrieved from Relational database
  - Updates job status → Stores in relational database
  - Uploads completion proof → Stored in File Storage

- Property Manager Interactions
  - Assigns technician → Updates Relational database
  - Monitors maintenance progress → Reads from Relation
  - Generates reports → Uses stored data

- System Processing
  - Maintenance tracking system → Updates request status
  - Notification system → Sends alerts (job assigned, completed, delayed)
  - Optional IoT integration → Detects faults (e.g., water leaks, power issues) and triggers automatic requests

3. Major Software Modules DFD (Level 2 DFD)

- User Authentication Module

- Input: User login credentials
- Process: System Authentication
- Output: Access token and user session

- Maintenance Request Management Module
  - Input: Maintenance issue detail (description, images, location)
  - Process: Store request in Relational database and assign priority
  - Output: Maintenance request ID and status

- Work Order and Assignment Module
  - Input: Maintenance request
  - Process: Assign service provider based on availability
  - Output: Work order assigned to service provider

- File Upload and Processing Module
  - Input: Images of maintenance issues
  - Process: Upload and store in cloud storage
  - Output: File URL linked to maintenance request

4. Internal Software Module DFD (Level 3 DFD)

- Maintenance Tracking Algorithm

Input:

- Maintenance request details
- Service provider updates
- Priority

Processing:

- Update request status (pending → in progress → completed)
- Track response time and completion time
- Log all actions in system

Output:

- Updated maintenance status
- Notifications to users

- Maintenance history records

Database Design (ERD)

1. Relation Database (SQL) Design

1.1 Tables and Data Dictionary

1.1.1 Tenant Table

Fields:

- Tenant_id (INT, Primary Key)
- Name (VARCHAR)
- Email (VARCHAR)
- Phone (VARCHAR)

Relationships:

- One-to-Many with Maintenance Requests

1.1.2 Property Table

Fields:

- Property_id (INT, Primary Key)
- Address (VARCHAR)
- Owner_id (INT, Foreign Key)

Relationships:

- Many-to-one with Owner
- One-to-many with maintenance requests

1.1.3 Property manager Table

Fields:

- Property_manager_id (INT, Primary Key)
- Name (VARCHAR)
- Contact (VARCHAR)

Relationships:

- One-to-many with property

1.1.4 Maintenance Request Table

Fields:

- Request_id (INT, Primary Key)
- Tenant_id (INT, Foreign Key)
- Property_id (INT, Foreign Key)

- Issue_description (TEXT)
- Priority (ENUM: pending, in_progress, completed)
- Created_at (TIMESTAMP)

  Relationships:
- Many-to-one with Tenant
- Many-to-one with Property
- One-to-one/ One-to-many with Work Order

1.1.5 Service Provider Table

Fields:

- Service_provider_id (INT, Primary Key)
- Name (VARCHAR)
- Skill (VARCHAR)
- Contact (VARCHAR)

  Relationships:
- One-to-many with Work Orders

1.1.6 Work Order Table

Fields:

- Work_id (INT, Primary Key)
- Request_id (INT, Foreign Key)
- Service_provider_id (INT, Foreign Key)
- Status (ENUM: assigned, in_progress, completed)
- Completion_date (TIMESTAMP)

  Relationships:
- Many-to-one with Maintenance request
- Many-to-one with Service provider

1,2 **Schema and Query Language**

- Uses Structured Query Language (SQL)
- Supports:
  - SELECT (retrieve maintenance records)
  - INSERT (new requests)
  - UPDATE (status changes)
  - DELETE (remove records if needed)

Keys and indexes:

- Primary keys: tenant_id, property_id, request_id

- Foreign keys: link tenants, properties, service providers
- Indexes on: email, status, timestamps

2.1**Query Language and Data Management**

Query Language:

The Smart Property Maintenance Tracker system utilizes SQL to manage structured and unstructured data efficiently

SQL Query Language:

- The relational database uses Structured Query Language (SQL) for data definition, manipulation, and querying
- Used for operation such as:
    - SELECT → retrieve maintenance requests, property data, and user details
    - INSERT → add new maintenance requests, users, and properties
    - UPDATE → modify request status (e.g., pending → completed)
    - DELECT → remove outdated or invalid records
    - JOIN operations → combine data from multiple tables (e.g., tenant + property + request)

Schema Flexibility:

- SQL Database:
    - Uses fixed schema with predefined tables and relationships
    - Ensures data consistency and integrity

Indices:

- Primary keys:
    - Tenant_id, property_id, request_id, service_provider_id
- Foreign keys:
    - Links between tenants, properties, maintenance requests, and service providers
- Custom indexing:

- Applied to frequently queried fields such as:
  - Email (for login)
  - Status (for filtering maintenance requests)
  - Timestamps (for sorting recent activity)

Data Management Functions:

Transaction:

- Ensures ACID properties (Atomicity, Consistency, Isolation, Durability)
- Used when:
  - Creating a maintenance request
  - Assigning a service provider
  - Updating request status

Stored Procedures / Triggers:

- Used to automate system processes such as:
  - Automatically updating maintenance status
  - Logging system activities
  - Sending alerts when a request is completed

Backup and Recovery:

- Daily automated backups of SQL databases
- File storage replication ensures data recovery in case of failure

Data Validation:

- Ensures only valid data is stored:
  - Required fields (e.g., issue description cannot be empty)
  - Correct formats (emails, phone number)
  - File validation for uploads (images, PDFs only)

3.Itegration Between System Components

System Integration

The Smart Property Maintenance Tracker system integrates the following components:

- Frontend (User Interface)
- Backend (Application Logic)
- Relational Database
- File Storage System

Data Flow

The backend acts as the central component that manages communication between the system components.

- Maintenance requests are created through the frontend and stored in the relational database
- Service providers update job status, which is recorded in the relational database
- Users retrieve updated information from the database through the backend
- Uploaded files are stored in the file storage system and linked to maintenance records

System Interaction Flow

- Frontend → Backend → Relational Database
- Frontend → Backend → File Storage
  Benefits of Integration
- Ensures efficient communication between system components
- Maintains data consistency and integrity
- Supports smooth system operation and scalability

FILES

/SmartPropertyMaintenanceTracker

/SmartPropertyMaintenanceTracker

```
├── /frontend
|   ├── /public
|   |   └── index.html
|   ├── /src
|   |   ├── /components
|   |   |   ├── Header.js
```

```
|   |   |   ├── Footer.js
|   |   |   ├── RequestForm.js
|   |   |   ├── RequestList.js
|   |   |   └── RequestDetails.js
|   |   ├── App.js
|   |   └── index.js
|   └── package.json
|
├── /backend
|   ├── /src
|   |   ├── server.js
|   |   ├── /routes
|   |   |   └── apiRoutes.js
|   |   ├── /controllers
|   |   |   └── requestController.js
|   |   └── /models
|   |       ├── userModel.js
|   |       ├── requestModel.js
|   |       └── propertyModel.js
|   └── config.js
|
├── /database
|   ├── schema.sql
|   └── /migrations
|       └── migration.sql
|
├── /storage
|   ├── /images
|   └── /documents
|
├── /docs
|   ├── SRS.docx
|   ├── SDD.docx
|   └── ProjectDocumentation.docx
|
└── .env
```

2. File Summary and Descriptions

File/Folder	Location	Description & Format
index.html	/frontend/public/	Main HTML file initializing the web app (HTML).
App.js / index.js	/frontend/src/	Core frontend logic and UI rendering (JavaScript, JSX).
package.json	/frontend/	Frontend dependencies and scripts (JSON).
server.js	/backend/src/	Backend server initialization (JavaScript).
apiRoutes.js	/backend/src/routes/	API endpoints for system operations (JavaScript).
Models	/backend/src/models/	Database schema definitions (JavaScript ORM).
requestController.js	/backend/src/controllers/	Business logic for maintenance processing (JavaScript).
config.js	/backend/	Configuration settings and environment setup (JavaScript).
schema.sql	/database/	Defines database tables and relationships (SQL).
migration_001.sql	/database/migrations/	Database updates and changes (SQL).
/images	/storage/images/	Stores maintenance images (JPG, PNG).
/videos	/storage/videos/	Stores maintenance videos (MP4).
/documents	/storage/documents/	Stores reports and invoices (PDF, DOCX).
.env	/	Environment variables and secrets (ENV).

Data and Configuration Files

Environment Variables

Stored in a .env file at the root of the project:

- DATABASE_URL
- API_KEYS
- AUTH_SECRET

Configuration Management

- Config.js file in the backend handles system configuration
- Loads environment variables and manages database connections\
- Defines API endpoints and system settings

Storage Files

- Images s of maintenance issues stored in / storage / images and / storage
- Documents such as invoices and reports stored in / storage / documents
- Each file is linked to a specific maintenance request

System Parameters

1. **Performance Parameters**
    - Optimized database queries using indexing
    - Catching frequently accessed data (e.g., maintenance history)
    - Efficient file retrieval from storage
2. **Security Parameters**
    - Encryption (TLS for data in transit, AES-256 for data at rest)
    - Role-Based Access Control (Tenant, Owner, Technician)
    - Secure authentication tokens
3. **Deployment Parameters**
    - Standard server-based deployments environment
    - Scalable system design
    - Secure API endpoints
4. **Data Management Parameters**
    - Daily database backups
    - Data retention policies for logs and files
    - File size limits for upload (e.g., max 10MB per file)

6. SYSTEM INTERFACES

6.1 User application Interface (Mobile & Web Dashboard)

Description

This interface defines the interaction between the end users (Tenants, Property managers, Service Providers, and System Administrators) and the Smart Property Maintenance Tracker (SMPT) system through mobile and web applications.

The interface enables users to perform system operations such as registrations, login/out, ticket submission, ticket monitoring, service provider assignment, and reporting. It ensures role-based access control and secure interaction with backend services.

Software Interface

- Frontend technologies:
  - React Native (mobile application)
  - React.js (web application)
- Backend communication:
  - RESTful API via node.js
- Protocols:
  - HTTPS (TLS 1.2 or higher)
- Authentication:
  - JWT (JSON Web Tokens)
- Data formats:
  - JSON for request and response handling

Hardware Interface

- Smartphones (Android or IOS)

- Desktop/Laptop computers
- Device components:
    - Camera (image capture)
    - GPS (Location Tracking)
    - Keyboard (data input)
    - Display screen

6.2 Authentication and Authorisation Interface

Description

This interface manages secure user authentication and authorisation across all systems users. It ensures that only verified and approved users can access the system based on their assigned roles.

It supports login/out, registration, email verification, password reset, account locking, and role- based access control.

Software Interface

- Node.js Authentication API
- JWT for session management
- AWS SES for email verification and password reset
- Encryption protocols for password hashing (e.g bycrypt)

Hardware Interface

- User devices (mobile or desktop)
- Network infrastructure supporting secure communication

6.3 AI Model integration Interface

Description

This interface enables communication between SPMT system and AI services used for ticket classification, image analysis, and emergency detection.

The system uses AI to automate ticket categorisation, detect conflicts, identify emergencies, and improve routing efficiency.

Software Interface

- AWS Comprehend (text classification)
- AWS Rekognition (image analysis)
- REST API calls to AWS services
- JSON data exchange format
- AI interface login system

Hardware Interface

- AWS cloud infrastructure (af-south-1 region)
- User device camera (for image capture)

6.4 Cloud storage and File Management Interface

Description

This interface manages the storage, retrieval, and management of images and files associated with maintenance tickets.

It ensures secure and scalable storage of attachments such as uploaded images and generated reports.

Software Interface

- AWS S3 for object storage
- Secure API integration with backend
- File upload/download using HTTPS
- Metadata management for file tracking

Hardware Interface

- Cloud storage servers (AWS infrastructure)
- User devices for uploading/downloading files

6.5 Notification and Messaging Interface

Description

This interface handles all communication between the system and users, including alerts, updates, and emergency notifications.

Notifications are triggered by system events as ticket creation, assignment, status updates, and SLA breaches.

Software Interface

- AWS SNS (SMS notifications)
- AWS SES (email notifications)
- Push notifications services
- REST API triggers from backend
- Message formats: JSON

Hardware Interface

- Mobile devices (for push notifications & SMS)
- Email servers
- Network communication systems

{abc Interface}

1. Ticket Processing and AI Classification Interface

Description

- This interface manages the flow of maintenance ticket data between the user application and the AI Engine. It is responsible for processing ticket submissions, triggering AI classification, detection conflicts, and generating classifications results.

Triggered events

- Tenant submits a maintenance ticket
- Images are uploaded with ticket data

Message Protocol/Data format

- JSON request:
  - TicketsID
  - Description

    - Images (URLs from S3)
    - Urgency level
- JSON response:
    - Category
    - Confidence score
    - Detected issues
    - Emergency flag

Hardware Interface

- AWS AI servers
- User devices (input source)

Error Handling / Recovery

- Retry mechanism for failed AI calls
- Fallback to manual classification if AI fails
- Logging of all errors in AI interface logs

Timing / Frequency

- AI processing must complete within 5 seconds
- Triggered per ticket submission

Security

- Encrypted API calls (HTTPS)
- IAM roles for AWS service access

2. Ticket Assignment and Routing Interface

Description

- This interface manages the assignment of classified tickets to suitable service providers based on category, proximity (GPS), and provider availability.

Triggering Events

- Successful AI classification of a ticket
- Manual override by property manager

Message Protocol / Data Format

- Input:
    - Ticket category
    - Provider list

    - GPS location
- Output:
    - Assigned provider ID
    - Assignment confirmation

Hardware Interface

- GPS-enabled mobile device
- Cloud servers for processing routing logic

Error Handling / Recovery

- Reassignment if provider declines
- Notification to manager for manual intervention

Timing / Frequency

- Immediate after classification
- Real-time processing

Security

- Role-based access control
- Secure API communication

3. Notification Delivery Interface
   Description
    - This interface ensures delivery of notifications to users regarding ticket updates, emergencies, and system alerts.

Triggering Events

- Ticket creation
- Ticket assignment
- Status updates
- Emergency detection
- SLA breaches

Message protocol / Data Format

- JSON message payload:
    - userID
    - Message content

- Notification types (SMS, Email, Push)

Hardware Interface

- Mobile devices (SMS, Push)
- Email servers

Error Handling / Recovery

- Retry delivery mechanism
- Queuing for failed messages
- Logging failed deliveries

Timing / Frequency

- Emergency notification: within 30 seconds
- Standard notification: within 10 minutes

Security

- Encrypted communication
- User authentication before message delivery

7. SYSTEM PERFORMANCE

7.1 Capacity and Sizing Calculations | File and database sizes

7.1.1 File Storage Capacity (Ticket Attachments)

The Smart Property Maintenance Tracker allows tenants to upload images when submitting maintenance tickets. Each ticket supports a maximum of five images, with each image limited to 10 MB.

For estimation purposes, the average image size is assumed to be 5 MB, with an average of 3 images uploaded per ticket. Therefore, the average storage per ticket is approximately 15 MB.

Assuming the system processes approximately 5,000 tickets per month:

- Monthly storage requirement = 5,000 × 15 MB = 75 GB
- Annual storage requirement = 75 GB × 12 = 900 GB (approximately 1 TB)

Therefore, the system must support large-scale storage of approximately 1 TB per year. To accommodate this growth, cloud storage (AWS S3) with automatic scalability will be used.

7.1.2 Database Storage Estimation (Structured Data)

The system stores structured data such as users, tickets, properties, notifications, and AI logs in a relational database.

Estimated storage requirements are as follows:

- Users: 10,000 records × 1 KB ≈ 10 MB
- Tickets: 100,000 records × 2 KB ≈ 200 MB
- Properties and Units: 5,000 records × 1 KB ≈ 5 MB
- Notifications: 200,000 records × 0.5 KB ≈ 100 MB
- AI Logs: 300,000 records × 1 KB ≈ 300 MB

Total estimated database size is approximately 615 MB. Including indexing, backups, and future growth, the system should provision at least 1 GB initially, with scalability up to 5–10 GB over time.

7.1.3 Unstructured Data Storage (Logs and Analytics)

Unstructured data includes AI inference logs, system activity logs, and analytics data. These data types grow continuously and may require between 500 MB to 2 GB per year.

7.2 System Limits

The system is designed to handle the following operational limits:

- **Concurrent Users:** Minimum of 500 users, scalable to over 5,000 users
- **API Requests**: 100–200 requests per second under normal load, scalable up to 500 requests per second during peak usage
- **Database Queries:** Average response time less than 100 milliseconds per query
- **Concurrent Database Operations**: At least 1,000 simultaneous queries supported

These limits ensure that the system remains stable and responsive even during peak usage periods.

7.3 Expected Response Times

The system is expected to meet the following performance targets:

- User Authentication (Login): Less than 1 second
- Ticket Submission: Less than 2 seconds
- Dashboard Loading: Less than or equal to 2 seconds
- AI Classification (text and image processing): Less than or equal to 5 seconds
- File Upload (images up to 10 MB): Between 3 to 5 seconds
- Emergency Notifications: Delivered within 30 seconds
- Standard Notifications: Delivered within 10 minutes

- Real-time updates (status changes): Between 200–500 milliseconds

These response times ensure a smooth and efficient user experience.

7.4 Performance-Related Executable Architectures

7.4.1 Cloud Infrastructure and Auto-Scaling

The system is deployed on AWS cloud infrastructure. Auto-scaling mechanisms and load balancers are used to dynamically allocate resources based on demand, ensuring high availability and efficient handling of traffic spikes.

7.4.2 AI Processing Architecture

The system integrates AWS Comprehend for text analysis and AWS Rekognition for image processing. AI tasks are executed asynchronously and in parallel to ensure that classification is completed within the required time limit of 5 seconds.

7.4.3 Caching Mechanisms

Redis is used as an in-memory caching system to store frequently accessed data such as dashboards and ticket summaries. This reduces database load and improves response times.

7.4.4 Content Delivery Network (CDN)

A CDN such as AWS CloudFront or Cloudflare is used to deliver static content (e.g., images) efficiently. This reduces latency and improves performance for users accessing the system from different locations.

7.4.5 Database Architecture

The system uses a hybrid database architecture:

Relational Database (SQL Server) for structured data

NoSQL or cloud logging systems for unstructured and high-volume data

This approach optimizes performance based on data type and access Hii patterns.

7.5 Calculation Examples

To guide future scalability planning, the following formulas are used:

- **File Storage Capacity** = Average File Size × Number of Files
- **Database Size** = Sum of (Number of Records × Average Record Size)
- **Response Time** = Network Latency + Processing Time + Database Query Time

For example, a typical response time may be calculated as:

50 ms (network latency) + 100 ms (processing) + 50 ms (database query) = 200 ms per operation.

7.6 Performance Summary

The Smart Property Maintenance Tracker is designed to support high system performance through scalable cloud infrastructure, efficient data storage, and optimized processing mechanisms. The system can handle large volumes of data (approximately 1 TB of file storage annually), maintain fast response times (less than 2 seconds for most operations), and process AI-based ticket classification within 5 seconds.

Additionally, the system ensures high availability, reliability, and scalability, making it suitable for real-world deployment across multiple properties and users.

8.GLOSSARY / TERMINOLOGY

9.REFERENCES

[1] **AWS (Amazon Web Services)** – Cloud computing, database management, API integratio
https://aws.amazon.com/rds/
https://aws.amazon.com/s3/

[2] **OpenAI (ChatGPT)** – AI-assisted responses, structuring of technical content, and clarification of concepts.

[3]

[…]